

Hashishcorp Consul ServiceMesh - Deployment, Monitoring and AlertManager(Zabbix/Nagios) configuration

Step-by-Step Runbook

HashiCorp Consul on OpenShift | UWM Stack | Zabbix 7.0

Field	Value
OpenShift Cluster	Version: 4.18.42
Helm	v3.19.0
Consul namespace	consul
Demo App Namespace	demo
Monitoring Namespace	monitoring (Zabbix VM)
Integration Method	Prometheus api
Mesh Version	Consul 1.20.5-ubi on consul-k8s Helm

1. Purpose

Purpose of this document is to configure the monitoring of Hashishcorp consul servicemesh and alerting receiver to Zabbix/Nagios UI

2. Scope

Scope of this document is to deploy

- Hashishcorp consul deployment
- Configuring Hashishcorp consul Matrix to default OCP UWM
- Integrate OCP default alertmanager with Zabbix/Nagios
- Display alerts in Zabbix/Nagios UI

3. Pre-Requisites

- OpenShift Cluster
- Helm
- Hashishcorp helm repo
- Thanos endpoint
- Redhat registry account secret
- Images required:

- hashicorp/consul:1.20.5-ubi, hashicorp/consul-k8s-control-plane:1.6.1-ubi, hashicorp/consul-dataplane:1.6.1-ubi from registry.connect.redhat.com/hashicorp/consul:1.20.5-ubi image

4. Deployment Steps for consul and test application

4.1 Create consul namespace:

```
oc new-project consul
```

4.2 Create redhat registry account secret - (because helm cannot use the OCP's pull-secret data)

```
oc create -f 15490118-openshift-secret.yml --namespace=consul
```

4.3 Download the required images - (consul, consul-dataplane and consul-k8s-control-plane)

```
[demo-user@bastion ~]$ oc import-image hashicorp/consul:1.20.5-ubi
--from=registry.connect.redhat.com/hashicorp/consul:1.20.5-ubi --confirm

[demo-user@bastion ~]$ oc import-image
hashicorp/consul-k8s-control-plane:1.6.1-ubi
--from=registry.connect.redhat.com/hashicorp/consul-k8s-control-plane:1.6.1-ubi
--confirmoc

[demo-user@bastion ~]$ oc import-image hashicorp/consul-dataplane:1.6.1-ubi
--from=registry.connect.redhat.com/hashicorp/consul-dataplane:1.6.1-ubi --confirm
```

4.4 create values.yaml

```
global:
  name: consul
  datacenter: dc1
  image: registry.connect.redhat.com/hashicorp/consul:1.20.5-ubi
  imagePullSecrets:
    - name: 19348389-jayatest-pull-secret
  openshift:
    enabled: true
  metrics:
    enabled: true
    enableAgentMetrics: true
    agentMetricsRetentionTime: "1m"
server:
```

```
replicas: 3
bootstrapExpect: 3
disruptionBudget:
  enabled: true
  maxUnavailable: 0
client:
  enabled: true
ui:
  enabled: true
  service:
    type: ClusterIP
metrics:
  enabled: true
  provider: "prometheus"
  baseURL: http://prometheus-server.consul.svc.cluster.local
dns:
  enabled: true
connectInject:
  enabled: true
  default: false
transparentProxy:
  defaultEnabled: true
metrics:
  defaultEnabled: true
  defaultEnableMerging: true
cni:
  enabled: true
  logLevel: info
  multus: true
  cniBinDir: /var/lib/cni/bin
  cniNetDir: /etc/kubernetes/cni/net.d
```

4.5 Deploy consul

```
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update
helm install consul hashicorp/consul --values values.yaml --create-namespace
--namespace consul --version "2.0.0" --wait
```

4.6 Expose the route

```
oc expose svc/consul-ui -n consul
```

```
oc get routes  
<consul-ui-consul.apps.cluster-2rs4f.2rs4f.sandbox3298.opentlc.com>
```

4.7 Test application deployment

Apply the following frontend and backend yaml and its services

frontend.yaml

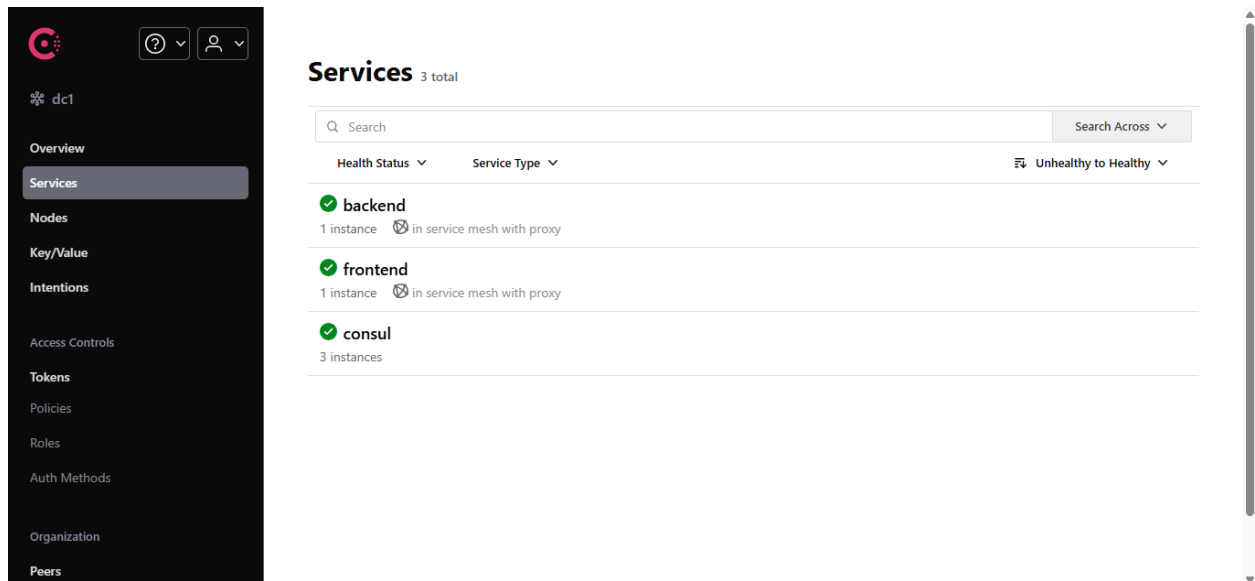
```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: frontend  
  namespace: demo  
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: frontend  
  template:  
    metadata:  
      labels:  
        app: frontend  
      annotations:  
        consul.hashicorp.com/connect-inject: "true"  
    spec:  
      containers:  
        - name: frontend  
          image: curlimages/curl  
          command: ["/bin/sh", "-c"]  
          args:  
            - while true; do curl http://backend:5678; sleep 5; done  
apiVersion: v1  
kind: Service  
metadata:  
  name: frontend  
  namespace: demo  
spec:  
  selector:  
    app: frontend  
  ports:  
    - port: 80  
      targetPort: 80
```

Backend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
      annotations:
        consul.hashicorp.com/connect-inject: "true"
    spec:
      containers:
        - name: backend
          image: hashicorp/http-echo
          args:
            - "-text=Hello from backend"
          ports:
            - containerPort: 5678
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: demo
spec:
  selector:
    app: backend
  ports:
    - port: 5678
```

targetPort: 5678

Consul UI



5. For test case we will deploy a prometheus in consul namespace

The following yaml is applied to create all requirements for the consul prometheus

```
---
# 1. ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus
  namespace: consul

---
# 2. Custom SCC for Prometheus - no anyuid needed
apiVersion: security.openshift.io/v1
kind: SecurityContextConstraints
metadata:
  name: prometheus-scc
allowPrivilegedContainer: false
allowPrivilegeEscalation: false
```

```
allowedCapabilities: []
defaultAddCapabilities: []
requiredDropCapabilities:
  - ALL
allowHostDirVolumePlugin: false
allowHostIPC: false
allowHostNetwork: false
allowHostPID: false
allowHostPorts: false
readOnlyRootFilesystem: false
runAsUser:
  type: RunAsAny
fsGroup:
  type: RunAsAny
seLinuxContext:
  type: MustRunAs
supplementalGroups:
  type: RunAsAny
volumes:
  - configMap
  - persistentVolumeClaim
  - projected
  - secret
  - emptyDir
users:
  - system:serviceaccount:consul:prometheus
groups: []

---
# 3. ClusterRole - permissions to scrape metrics
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
  - apiGroups: [""]
    resources:
      - nodes
      - nodes/proxy
      - services
      - endpoints
      - pods
    verbs: ["get", "list", "watch"]
```

```

- apiGroups: ["extensions", "networking.k8s.io"]
  resources:
    - ingresses
  verbs: ["get", "list", "watch"]
- nonResourceURLs: ["/metrics"]
  verbs: ["get"]

---
# 4. ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus
subjects:
- kind: ServiceAccount
  name: prometheus
  namespace: consul

---
# 5. ConfigMap - Prometheus scrape config
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
  namespace: consul
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
      evaluation_interval: 15s

    scrape_configs:

      # Scrape Consul servers
      - job_name: 'consul-server'
        static_configs:
          - targets:
              - consul-server-0.consul-server.consul.svc.cluster.local:8500
              - consul-server-1.consul-server.consul.svc.cluster.local:8500

```



```

        - consul-server-2.consul-server.consul.svc.cluster.local:8500
metrics_path: /v1/agent/metrics
params:
  format: ['prometheus']

# Scrape Consul clients
- job_name: 'consul-client'
static_configs:
  - targets:
    - consul-client.consul.svc.cluster.local:8500
metrics_path: /v1/agent/metrics
params:
  format: ['prometheus']

# Scrape Envoy sidecars via pod annotation discovery
- job_name: 'consul-mesh-envoy'
kubernetes_sd_configs:
  - role: pod
relabel_configs:
  - source_labels:
    [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
    action: keep
    regex: "true"
  - source_labels:
    [__meta_kubernetes_pod_annotation_prometheus_io_path]
    action: replace
    target_label: __metrics_path__
    regex: (.+)
  - source_labels: [__address__],
    [__meta_kubernetes_pod_annotation_prometheus_io_port]
    action: replace
    regex: ([^:]+)(?::\d+)?;(\d+)
    replacement: $1:$2
    target_label: __address__
  - source_labels: [__meta_kubernetes_namespace]
    target_label: namespace
  - source_labels: [__meta_kubernetes_pod_name]
    target_label: pod

---
# 6. PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim

```

```

metadata:
  name: prometheus-data
  namespace: consul
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi

---
# 7. Deployment - fully OCP native, no hardcoded UIDs
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus-server
  namespace: consul
  labels:
    app: prometheus
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
      annotations:
        consul.hashicorp.com/connect-inject: "false"
    spec:
      serviceAccountName: prometheus
      containers:
        - name: prometheus
          image: prom/prometheus:v2.51.0
          args:
            - "--config.file=/etc/prometheus/prometheus.yml"
            - "--storage.tsdb.path=/prometheus"
            - "--storage.tsdb.retention.time=15d"
            -
            --web.console.libraries=/usr/share/prometheus/console_libraries"
            - "--web.console.templates=/usr/share/prometheus/consoles"
            - "--web.enable-lifecycle"

```

```
    ports:
      - containerPort: 9090
        name: web
    volumeMounts:
      - name: config
        mountPath: /etc/prometheus
      - name: data
        mountPath: /prometheus
    resources:
      requests:
        cpu: 200m
        memory: 512Mi
      limits:
        cpu: 500m
        memory: 1Gi
    readinessProbe:
      httpGet:
        path: /-/ready
        port: 9090
        initialDelaySeconds: 10
        periodSeconds: 10
    livenessProbe:
      httpGet:
        path: /-/healthy
        port: 9090
        initialDelaySeconds: 30
        periodSeconds: 15
  volumes:
    - name: config
      configMap:
        name: prometheus-config
    - name: data
      persistentVolumeClaim:
        claimName: prometheus-data
```

8. Service

apiVersion: v1

kind: Service

metadata:

name: prometheus-server

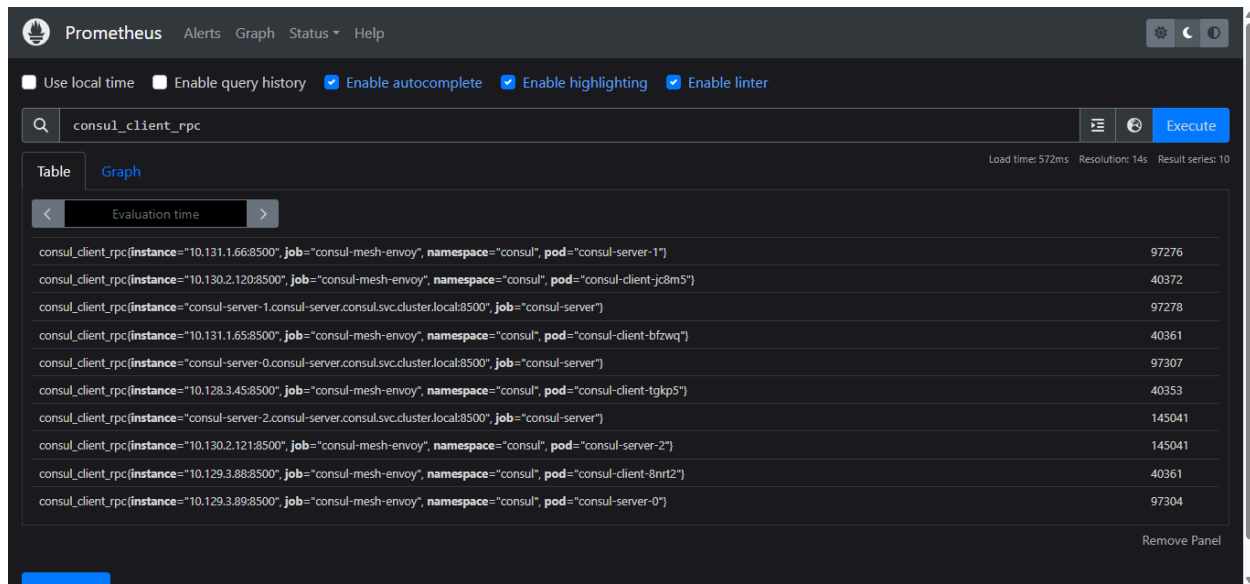
namespace: consul

labels:

```
    app: prometheus
spec:
  selector:
    app: prometheus
  ports:
    - port: 80
      targetPort: 9090
      name: web

---
# 9. Route - expose Prometheus UI
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: prometheus-ui
  namespace: consul
spec:
  to:
    kind: Service
    name: prometheus-server
  port:
    targetPort: web
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

Prometheus UI



6. Create servicemonitor to configure service mesh to OCP UWM

apply the following yaml to configure the matrix from consul to OCP UWM

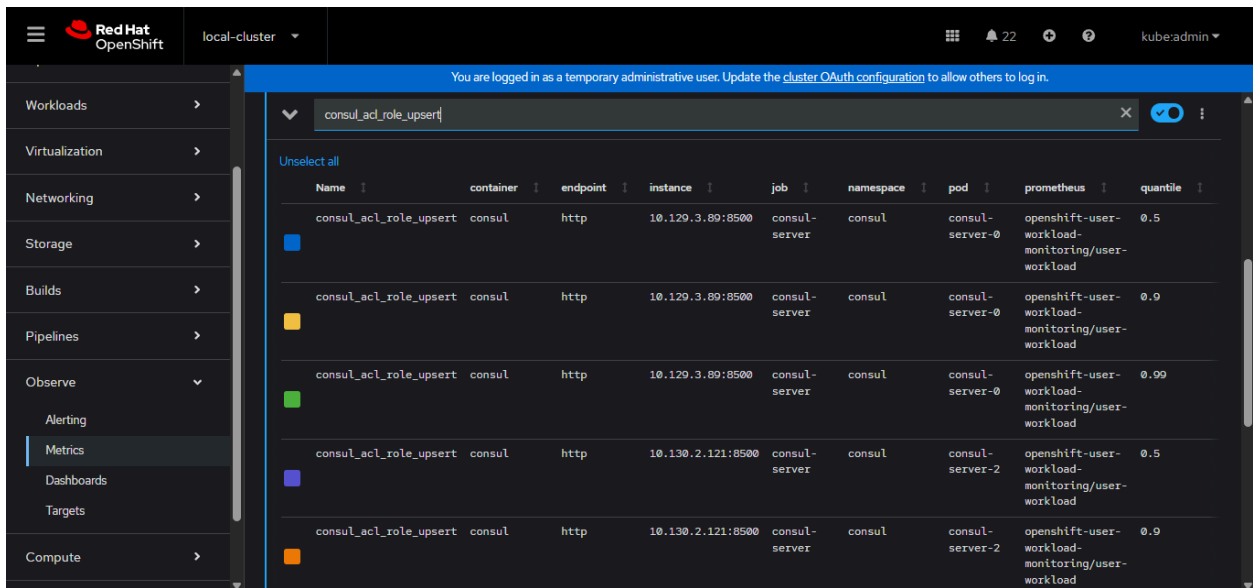
```
# ServiceMonitor for Consul servers
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: consul-server
  namespace: consul
  labels:
    app: consul
spec:
  endpoints:
    - port: http
      path: /v1/agent/metrics
      params:
        format: ['prometheus']
      interval: 15s
  namespaceSelector:
    matchNames:
      - consul
  selector:
    matchLabels:
      component: server
---
```

```
# ServiceMonitor for Consul clients
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: consul-client
  namespace: consul
  labels:
    app: consul
spec:
  endpoints:
    - port: http
      path: /v1/agent/metrics
      params:
        format: ['prometheus']
        interval: 15s
  namespaceSelector:
    matchNames:
      - consul
  selector:
    matchLabels:
      component: client

---
# ServiceMonitor for Envoy sidecars in my-app
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: consul-mesh-envoy
  namespace: demo
  labels:
    release: prometheus
spec:
  namespaceSelector:
    matchNames:
      - demo
  selector:
    matchExpressions:
      - key: app
        operator: In
        values:
          - frontend
          - backend
  endpoints:
```

```
- port: prometheus
  path: /metrics
  interval: 15s
```

OCP observe UI



The screenshot shows the Red Hat OpenShift OCP observe UI. The left sidebar contains navigation links for Workloads, Virtualization, Networking, Storage, Builds, Pipelines, Observe, Alerting, Metrics, Dashboards, Targets, and Compute. The main content area displays a table of Prometheus metrics for the job 'consul_acl_role_upsert'. The table has columns for Name, container, endpoint, instance, job, namespace, pod, prometheus, and quantile. There are five rows of data, each with a colored square icon to its left. A blue banner at the top of the table area states: 'You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.'

Name	container	endpoint	instance	job	namespace	pod	prometheus	quantile
consul_acl_role_upsert	consul	http	10.129.3.89:8500	consul-server	consul	consul-server-0	openshift-user-workload-monitoring/user-workload	0.5
consul_acl_role_upsert	consul	http	10.129.3.89:8500	consul-server	consul	consul-server-0	openshift-user-workload-monitoring/user-workload	0.9
consul_acl_role_upsert	consul	http	10.129.3.89:8500	consul-server	consul	consul-server-0	openshift-user-workload-monitoring/user-workload	0.99
consul_acl_role_upsert	consul	http	10.130.2.121:8500	consul-server	consul	consul-server-2	openshift-user-workload-monitoring/user-workload	0.5
consul_acl_role_upsert	consul	http	10.130.2.121:8500	consul-server	consul	consul-server-2	openshift-user-workload-monitoring/user-workload	0.9

7. Zabbix deployment

We will be deploying zabbix in a VM which is running in ocp-v in the same cluster

7.1 apply the following vm.yaml and datavolume.yaml

```
apiVersion: kubevirt.io/v1
```

```
kind: VirtualMachine
metadata:
  name: zabbix-server
  namespace: monitoring
spec:
  running: true
  template:
    metadata:
      labels:
        kubevirt.io/domain: zabbix-server
    spec:
      domain:
        cpu:
          cores: 4
        resources:
          requests:
            memory: 8Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            dataVolume:
              name: zabbix-server-rootdisk

          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                hostname: zabbix-server

              users:
```



```

      - default
      - name: user_a
        groups: sudo
        sudo: ALL=(ALL) NOPASSWD:ALL
        shell: /bin/bash
        lock_passwd: false

    ssh_pwauth: true

    chpasswd:
      list: |
        user_a:Redhat@1234
      expire: false

    package_update: true
  apiVersion: cdi.kubevirt.io/v1beta1
  kind: DataVolume
  metadata:
    name: zabbix-server-rootdisk
    namespace: monitoring
  spec:
    source:
      http:
        url:
          https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.i
          mg
    pvc:
      accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: 100Gi
      storageClassName: ocs-storagecluster-ceph-rbd

```

```

[demo-user@bastion ~]$ oc get vm -n monitoring
NAME          AGE    STATUS    READY
zabbix-server 2d6h   Running   True
[demo-user@bastion ~]$

```

7.2 Once VM is up and running perform the following steps in VM to setup zabbix

7.2.1 Install docker.io docker-compose-plugin postgresql postgresql-contrib php-pgsql nginx php8.1-fpm

```
sudo apt update
apt install -y docker.io docker-compose-plugin postgresql
postgresql-contrib php-pgsql nginx php8.1-fpm
```

7.2.2 create a docker compose

```
cat <<EOF > docker-compose.yml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: zabbixpass
      POSTGRES_DB: zabbix
    volumes:
      - pg_data:/var/lib/postgresql/data

  zabbix-server:
    image: zabbix/zabbix-server-pgsql:latest
    environment:
      DB_SERVER_HOST: postgres
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: zabbixpass
      POSTGRES_DB: zabbix
    depends_on:
      - postgres

  zabbix-web:
    image: zabbix/zabbix-web-nginx-pgsql:latest
    environment:
      DB_SERVER_HOST: postgres
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: zabbixpass
      POSTGRES_DB: zabbix
      ZBX_SERVER_HOST: zabbix-server
      PHP_TZ: Asia/Kolkata
    ports:
      - "8080:8080"
    depends_on:
      - zabbix-server

volumes:
```

```
pg_data:
EOF
```

```
docker-compose up -d
```

7.2.3 Download zabbix repo and add them

```
wget
https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release\_7.0-2+ubuntu22.04\_all.deb

dpkg -i zabbix-release_7.0-2+ubuntu22.04_all.deb
```

7.2.4 Install zabbix-server-pgsql zabbix-frontend-php zabbix-nginx-conf zabbix-sql-scripts zabbix-agent

```
apt install -y zabbix-server-pgsql zabbix-frontend-php zabbix-nginx-conf
zabbix-sql-scripts zabbix-agent
```

7.2.5 Configure zabbix

```
sudo -u postgres psql
<INSIDE psql>
CREATE DATABASE zabbix;
CREATE USER zabbix WITH PASSWORD 'zabbixpass';
ALTER ROLE zabbix SET client_encoding TO 'utf8';
ALTER ROLE zabbix SET default_transaction_isolation TO 'read committed';
ALTER ROLE zabbix SET timezone TO 'UTC';
GRANT ALL PRIVILEGES ON DATABASE zabbix TO zabbix;
\q
<INSIDE qsql>
zcat /usr/share/zabbix-sql-scripts/postgresql/server.sql.gz | sudo -u
zabbix psql zabbix
    nano /etc/zabbix/zabbix_server.conf
<DBHost=localhost
DBName=zabbix
DBUser=zabbix
DBPassword=zabbixpass

nano /etc/zabbix/nginx.conf
server {
```

```
listen      8080;
server_name _;

root        /usr/share/zabbix;

index       index.php;

location = /favicon.ico {
    log_not_found    off;
}

location / {
    try_files         $uri $uri/ =404;
}

location /assets {
    access_log        off;
    expires           10d;
}

location ~ /\.ht {
    deny              all;
}

location ~ /(api\|conf[^\.]|include|locale) {
    deny              all;
    return            404;
}

location /vendor {
    deny              all;
    return            404;
}

location ~ [^/]\.php(/|$) {
    fastcgi_pass       unix:/var/run/php/zabbix.sock;
    fastcgi_split_path_info ^(.+\.php)(/.+)$;
    fastcgi_index      index.php;

    fastcgi_param      DOCUMENT_ROOT    /usr/share/zabbix;
    fastcgi_param      SCRIPT_FILENAME
/usr/share/zabbix$fastcgi_script_name;
    fastcgi_param      PATH_TRANSLATED
```

```

/usr/share/zabbix$fastcgi_script_name;

    include fastcgi_params;
    fastcgi_param    QUERY_STRING       $query_string;
    fastcgi_param    REQUEST_METHOD     $request_method;
    fastcgi_param    CONTENT_TYPE       $content_type;
    fastcgi_param    CONTENT_LENGTH     $content_length;

    fastcgi_intercept_errors            on;
    fastcgi_ignore_client_abort        off;
    fastcgi_connect_timeout            60;
    fastcgi_send_timeout               180;
    fastcgi_read_timeout               180;
    fastcgi_buffer_size                128k;
    fastcgi_buffers                    4 256k;
    fastcgi_busy_buffers_size          256k;
    fastcgi_temp_file_write_size      256k;
}
}

systemctl enable zabbix-server zabbix-agent nginx php8.1-fpm

```

7.3 Once the VM configuration is done expose the route by applying the following:

```

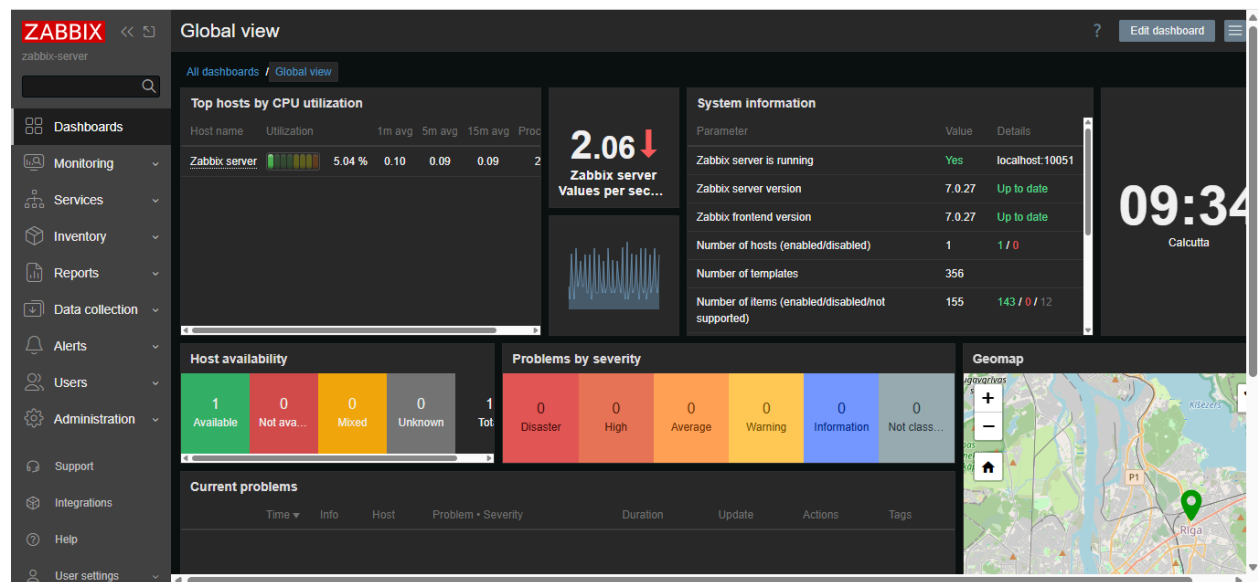
apiVersion: v1
kind: Service
metadata:
  name: zabbix-web
  namespace: monitoring
spec:
  selector:
    kubevirt.io/domain: zabbix-server
  ports:
    - name: http
      port: 8080
      targetPort: 8080
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: zabbix-web
  namespace: monitoring
spec:

```

```
to:
  kind: Service
  name: zabbix-web
port:
  targetPort: http
```

```
[demo-user@bastion ~]$ oc get route -n monitoring
NAME          HOST/PORT                                     PATH  SERVICES  PORT  TERMINATION  WILDCARD
zabbix-web    zabbix-web-monitoring.apps.cluster-2rs4f.2rs4f.sandbox3298.opentlc.com  PATH  zabbix-web  http  None
```

Zabbix web UI



Remark:

To configure AlertManager using Webhook for Zabbix, following updates:

1. **Issue** - the image being used for Webhook has port 8080 and the port we require to configure is 9095. After changing the ports in the Webhook yaml the connection got established but the alerts are not being forwarded.
2. From the Zabbix VM an alert was triggered which was received by Zabbix, which ensures the functionality of Zabbix is fine.

Solution

Using **Prometheus API** to integrate alerts with Zabbix

Procedure:

- Create a service account in openshift-monitoring namespace and assign cluster-monitoring-view role to that SA

```
oc create sa zabbix-sa
oc adm policy add-cluster-role-to-user cluster-monitoring-view -z zabbix-sa
```

- Create service account token and get thanos endpoint

```
oc create token zabbix-sa \
  -n openshift-monitoring \
  --duration=24h

oc get route thanos-querier -n openshift-monitoring
```

- Test

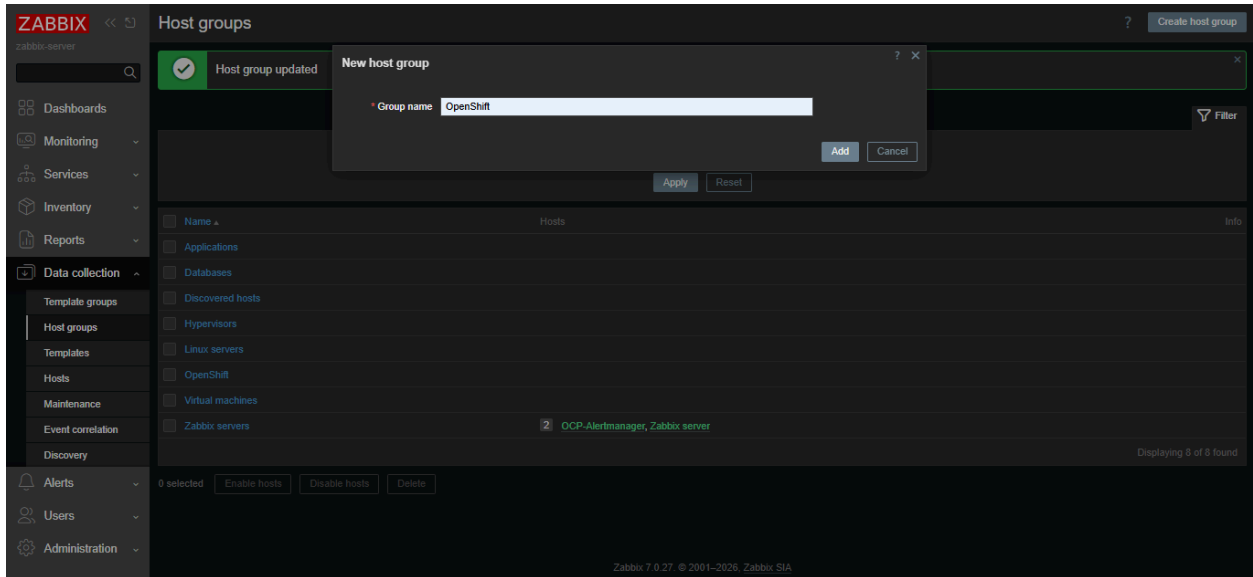
```
curl -sk -H "Authorization: Bearer $TEST"
"https://$THANOS/api/v1/query?query=ALERTS%7Bnamespace%3D~%22consul%7Cdemo%
22%2Calertstate%3D%22firing%22%7D"
```

Zabbix Template Setup

Create a host group

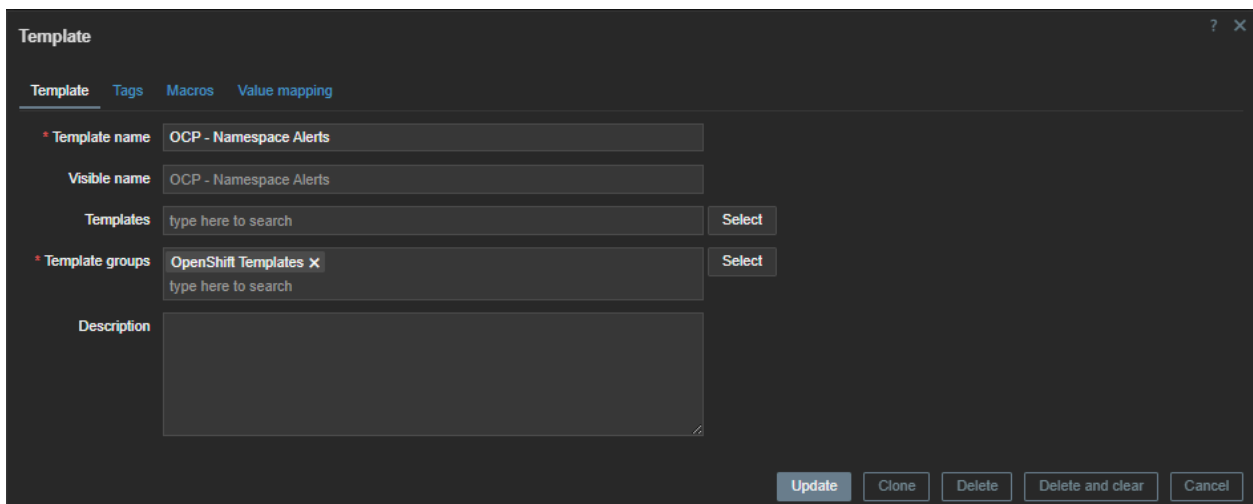
Host groups organise your Zabbix hosts. Create one dedicated to OpenShift monitoring.

1. In the Zabbix UI, navigate to Data collection → Host groups.
2. Click Create host group.
3. Enter "OpenShift" (or your preferred name) in the Group name field.
4. Click Add.



Create the monitoring template

5. Navigate to Data collection → Templates.
6. Click Create template.
7. Set Template name to "OCP - Namespace Alerts (consul/demo)".
8. In the Template groups field, type a group name (e.g. "OpenShift Templates") and select the new option that appears.
9. Click Add.



Create the Master HTTP Agent item

This item polls the Thanos API and stores the raw JSON response. The LLD rule and item prototypes depend on it.

Navigate to Data collection → Templates → select your template → Items → Create item. Fill in the fields below:

Field	Value	Notes
Name	Get OCP namespace alerts	Descriptive display name
Type	HTTP agent	API call over HTTPS
Key	prom.get.ns.alerts	Unique item identifier
Type of information	Text	Raw JSON payload
URL	{\${PROMETHEUS_OCP}}/api/v1/query	Macro populated on the host
Query fields — Name	query	PromQL parameter key
Query fields — Value	ALERTS{namespace=~"consul demo",alertstate="firing"}	Namespace filter
Request type	GET	Read-only query
Headers — Name	Authorization	Auth header key
Headers — Value	{\${TOKEN}}	Macro: Bearer <token>
Update interval	1m	Poll every minute
History storage	1h	Keep raw payload 1 hour only

Item

Item
Tags
Preprocessing 2

* Name
Get OCP namespace alerts

Type
HTTP agent

* Key
prom.get.ns.alerts
Select

Type of information
Text

* URL
{\${PROMETHEUS_OCP}}/api/v1/query
Parse

Query fields

Name
Value

query
⇒
ALERTS{namespace=~"consul|demo",
Remove

Add

Request type
GET

Request body type
Raw data
JSON data
XML data

Request body

Headers

Name
Value

Authorization
⇒
{\${TOKEN}}
Remove

Add

Required status codes
200

Follow redirects
☒

Update
Clone
Test
Delete
Cancel

Item
?
X

Item
Tags
Preprocessing 2

SSL verify peer ☐

SSL verify host ☐

SSL certificate file

SSL key file

SSL key password

* Update interval 1m

Custom intervals

Type	Interval	Period	Action
Flexible	Scheduling	50s	1-7,00:00-24:00
Remove			
Add			

* Timeout Global Override 3s Timeouts

* History Do not store Store up to 1d

Enable trapping ☐

Populates host inventory field -None-

Description

Enabled ☒

Update Clone Test Delete Cancel

Add preprocessing steps

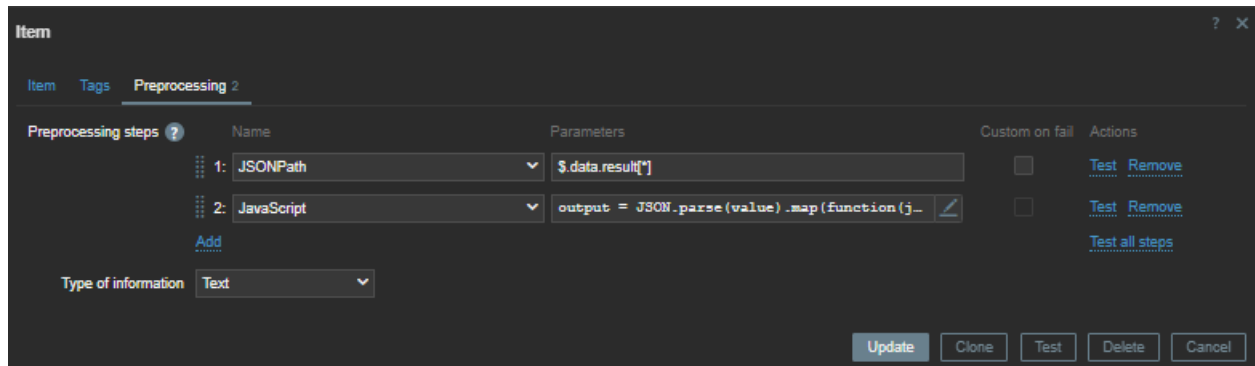
After filling in the item fields, click the Preprocessing tab and add two steps in order:

Step 1 — JSONPath (extracts the result array from the Prometheus response envelope):

```
$.data.result[*]
```

Step 2 — JavaScript (reshapes each result into a flat object with four named fields):

```
output = JSON.parse(value).map(function(js) {
  return {
    "alertname": js.metric.alertname,
    "namespace": js.metric.namespace,
    "severity": js.metric.severity || "unknown",
    "state": js.value[1]
  };
});
return JSON.stringify({ "data": output });
```



Test the preprocessing

10. On the Preprocessing tab click Test all steps.
11. Tick Get value from host.
12. Enter the Thanos URL in `{ $PROMETHEUS_OCP }` and your token (with Bearer prefix) in `{ $TOKEN }`.
13. Click Get value and test.
14. Confirm Result shows a JSON array with alertname, namespace, severity, and state fields. If data.result is empty, no alerts are currently firing — that is normal.

Field	Value	Notes
Keep lost resources	1d	Retain items 1 day after alert clears
Update interval	1m	Inherited from master item

Configure LLD macros

Click the LLD macros tab inside the discovery rule and add three macros that map to fields in the JavaScript output:

Macro	JSONPath expression	Purpose
{#ALERTNAME}	\$.alertname	Alert rule name
{#NAMESPACE}	\$.namespace	Source namespace (consul or demo)
{#SEVERITY}	\$.severity	Alert severity label

Create item prototypes

Inside the discovery rule, click Item prototypes → Create item prototype. LLD will instantiate one item per unique {#ALERTNAME}/{#NAMESPACE} combination.

Field	Value	Notes
Name	OCP alert: {#ALERTNAME} in {#NAMESPACE}	Dynamic name per alert
Type	Dependent item	Depends on master item
Key	ocp.alert[{#NAMESPACE},{#ALERTNAME}]	Unique per namespace+alert
Type of information	Numeric (unsigned)	State is 0 or 1

Field	Value	Notes
Master item	Get OCP namespace alerts	
Applications	OCF Alerts	Optional grouping tag

New item prototype

Item prototype

Tags

Preprocessing

* Name

OCF alert: {#ALERTNAME} in {#NAMESPACE}

Type

Dependent item

* Key

ocf.alert[{#NAMESPACE},{#ALERTNAME}]

Select

Type of information

Numeric (unsigned)

* Master item

OCF - Namespace Alerts: Get OCP namespace alerts X

Select

Select prototype

Units

* History

Do not store

Store up to

31d

* Trends

Do not store

Store up to

365d

Value mapping

type here to search

Select

Description

Create enabled

☒

Discover

☒

Add

Test

Cancel

Item prototype preprocessing

On the item prototype Preprocessing tab, add one JSONPath step to extract the state value for the specific alert:

\$.data[?(@.alertname=='{#ALERTNAME}' && @.namespace=='{#NAMESPACE}')].state.first()

New item prototype

Item prototype

Tags

Preprocessing 1

Preprocessing steps

1: JSONPath

\$data[?(@.alertname=='{#ALERTNAME}' && @.namespace=

Add

Type of information

Numeric (unsigned)

Add

Test

Cancel

Create trigger prototypes

Inside the discovery rule, click Trigger prototypes → Create trigger prototype.

Field	Value
Name	OCF alert {#ALERTNAME} firing in {#NAMESPACE} [{#SEVERITY}]
Severity	Map to Zabbix severity — see table below
Expression	last(/OCF - Namespace Alerts/ocp.alert[{#NAMESPACE},{#ALERTNAME}])<>0
Description	Alert {#ALERTNAME} is actively firing in namespace {#NAMESPACE}. Severity: {#SEVERITY}
Recovery exp	last(/OCF - Namespace Alerts/ocp.alert[{#NAMESPACE},{#ALERTNAME}])=0

New trigger prototype

Trigger prototype Tags Dependencies

* Name: OCF alert {#ALERTNAME} firing in {#NAMESPACE} [{#SEVERITY}]

Event name: OCF alert {#ALERTNAME} firing in {#NAMESPACE} [{#SEVERITY}]

Operational data:

Severity: Not classified Information Warning Average High Disaster

* Problem expression: last(/OCF - Namespace Alerts/ocp.alert[{#NAMESPACE},{#ALERTNAME}])<>0 Add

Expression constructor

OK event generation: Expression Recovery expression None

* Recovery expression: last(/OCF - Namespace Alerts/ocp.alert[{#NAMESPACE},{#ALERTNAME}])=0 Add

Expression constructor

PROBLEM event generation mode: Single Multiple

OK event closes: All problems All problems if tag values match

Allow manual close: ☐

Menu entry name: ? Trigger URL

Menu entry URL:

Description: Alert {#ALERTNAME} is actively firing in namespace {#NAMESPACE}. Severity: {#SEVERITY}

Add Cancel

Severity mapping

Prometheus severity label	Recommended Zabbix severity
critical	Disaster
warning	Average
info	Information

Prometheus severity label	Recommended Zabbix severity
none / unknown	Warning

Host Configuration in Zabbix

Create the host

The host represents your OpenShift cluster inside Zabbix and holds the macro values for the Thanos endpoint and bearer token.

1. Navigate to Data collection → Hosts → Create host.
2. Fill in the fields:

Field	Value
Host name	OCF - consul/demo Namespace Monitoring
Templates	OCF - Namespace Alerts (consul/demo)
Host groups	OpenShift
Interfaces	Agent — IP: 127.0.0.1, Port: 10050 (placeholder, not used for HTTP agent items)

Host

Host IPMI Tags Macros 2 Inventory Encryption Value mapping

* Host name: OCF - consul demo Namespace Monitoring

Visible name: OCF - consul demo Namespace Monitoring

Templates

Name	Action
OCF - Namespace Alerts	Unlink Unlink and clear

type here to search Select

* Host groups: OpenShift X Select

type here to search

Interfaces

Type	IP address	DNS name	Connect to	Port	Default
Agent	127.0.0.1		IP DNS	10050	☒ Remove

[Add](#)

Description

Monitored by: Server Proxy Proxy group

Enabled ☒

Update Clone Delete Cancel

Set host macros

Click the Macros tab on the host and add the two macros below. These override the template-level macros with cluster-specific values.

Macro	Value format	Example
<code>{\${PROMETHEUS_OCP}}</code>	<code>https://<thanos-querier-route></code>	<code>https://thanos-querier-openshift-monitoring.apps.cluster.example.com</code>
<code>{\${TOKEN}}</code>	<code>Bearer <token></code>	<code>Bearer eyJhbGciOiJSUzI1NiIsInR5cC...</code>

The screenshot shows the 'Host' configuration window with the 'Macros' tab selected. The interface includes tabs for Host, IPMI, Tags, Macros 2, Inventory, Encryption, and Value mapping. Under 'Host macros', there are two entries:

Macro	Value	Description
<code>{\${PROMETHEUS_OCP}}</code>	<code>https://thanos-querier-openshift-monitoring.apps.cluster-2rs4f.sandbox3298.opentlc.com</code>	description
<code>{\${TOKEN}}</code>	<code>Bearer eyJhbGciOiJSUzI1NiIsInR5cC...</code>	description

Buttons at the bottom include 'Add', 'Update', 'Clone', 'Delete', and 'Cancel'.

Validate host configuration

1. Click Update to save the host.
2. Wait 1–2 minutes for the first LLD execution.
3. Navigate to Data collection → Hosts → click Items on your host.
4. Confirm that items like "OCP alert: Watchdog in consul" appear (if alerts are firing).
5. Navigate to Monitoring → Latest data and filter by your host to confirm values are being collected.

All hosts / OCP consul demo Namespace Mo...

Enabled2Items2

Triggers1GraphsDiscovery rules1Web scenarios

Host groups

type here to search

Select

Hosts

OCP consul demo N... X

type here to search

Select

Name

type here to search

Key

type here to search

Value mapping

type here to search

Select

Type

All

Type of information

All

History

type here to search

Trends

type here to search

Update interval

type here to search

Tags

And/Or Or

tag

Contains

value

Remove

Add

State

All Normal Not supported

Status

All Enabled Disabled

Triggers

All Yes No

Inherited

All Yes No

Discovered

All Yes No

ApplyReset

Subfilter affects only filtered data

TYPES

Dependent item HTTP agent

TYPE OF INFORMATION

Numeric (unsigned) Text

TEMPLATE

Not inherited items Inherited items

WITH TRIGGERS

Without triggers With triggers

DISCOVERY

Regular Discovered

HISTORY

1d 1w

☐	Name	Triggers	Key	Interval	History	Trends	Type	Status	Tags	Info
☐	+++ OCP - Namespace Alerts: Get OCP namespace alerts		prom.get.ns.alerts	1m	1d		HTTP agent	Enabled		
☐	+++ LLD OCP namespace alerts: Get OCP namespace alerts: OCP alert: PodDisruptionBudgetAtLimit in consul	Triggers	ocp.alert[consul.PodDisruptionBudgetAtLimit]		7d	7d	Dependent item	Enabled		

Displaying 2 of 2 found

Severity

Name

Expression

Status

Not classified

OCP alert PodDisruptionBudgetAtLimit firing in consul [warning]

Problem: last(OCP consul demo Namespace Monitoring/ocp.alert[consul.PodDisruptionBudgetAtLimit])<=0
Recovery: last(OCP consul demo Namespace Monitoring/ocp.alert[consul.PodDisruptionBudgetAtLimit])=0

Enabled